BCS155 HTML & CSS Lab: Detailed Report

Invertis University, Bareilly. B.Tech CSE, First Year. Name: YOUR NAME. Roll No: YOUR ROLL NO.

This report explains all 15 lab assignments: the aim, the code, what each part does, and the output you should see. The complete code files are in the zip (folders a03 to a15). Replace the name and roll number placeholders before submitting, and take your own screenshots and terminal copies.

Assignment 1: Starting a Git Repository

Aim: Create a repository for the lab folder and save work with commits.

Key ideas. Git tracks changes in a folder. `git init` creates a hidden `.git` folder that stores history. A commit is a saved snapshot. A file moves through three stages: working folder, staging area (`git add`), and repository (`git commit`).

Task 1 commands

```
mkdir html-lab
cd html-lab
git init
git config --local user.name "Your Name"
git config --local user.email "you@example.com"
git config user.name
git config user.email
```

`--local` stores the name and email only for this repository, not for the whole computer. The last two commands print the values back, e.g. `Your Name` and `you@example.com`.

Task 2 commands

```
echo "<h1>Home</h1>" > index.html
git status
git add index.html
git commit -m "Add home page"
```

`git status` lists `index.html` under "Untracked files" because Git has not seen it before. `git add` stages it. `git commit -m` saves it with a message. Expected output ends with a line like `[main (root-commit) a1b2c3d] Add home page` and `1 file changed, 1 insertion(+)`.

Task 3 commands

```
echo "<h2>Next meeting</h2>" >> index.html
git add index.html
git commit -m "Add next meeting heading"
git log --oneline
```

`>>` appends to the file, so the h2 goes below the h1. `git log --oneline` shows one line per commit, newest first:

```
d4e5f6a Add next meeting heading
a1b2c3d Add home page
```

The short codes (hashes) will be different on your computer.

Assignment 2: Pushing and Cloning a Repository

Aim: Send commits to a second repository and copy it with clone.

Key ideas. A remote is another copy of the repository. A bare repository has no working files and is used only to receive pushes, like a server. `origin` is the usual name for the main remote. `clone` makes a full copy, including all history.

Task 1

```
git init --bare ../html-remote
git remote add origin ../html-remote
git remote -v
```

`../html-remote` means the folder next to html-lab. The last command prints two lines: `origin ../html-remote (fetch)` and `origin ../html-remote (push)`.

Task 2

```
git push origin main
```

If `git status` says `On branch master`, use `git push origin master`. The output ends with `* [new branch] main -> main`, which means your local branch is now on the remote.

Task 3

```
cd ..
git clone html-remote html-copy
cd html-copy
git log --oneline
```

The clone prints `Cloning into 'html-copy'...`. The log shows the same two commits as in html-lab, which proves the push and clone worked.

Assignment 3: Your First HTML Page

Aim: Write a page with a title, headings and a paragraph, and format text.

Key ideas. Every page has the structure `<!DOCTYPE html>`, `<html>`, `<head>` (information about the page, such as the title) and `<body>` (what the user sees). Headings run from h1 (largest) to h6 (smallest).

Task 1, first.html. The title is My First Page and the body has `<h1>Hello, I am Asha</h1>`. The browser tab shows the title; the page shows one large heading.

Task 2, headings.html. It uses `<h1>`, `<h2>`, `<h3>` and a `<p>`. Each heading is smaller than the one before it.

Task 3, text-format.html.

```
<p>This paragraph has a <strong>bold</strong> word.<br>This text starts on a
new line.</p>
<p>This paragraph has an <em>italic</em> word.</p>
```

`<strong>` makes text bold and marks it as important. `<em>` makes text italic and marks emphasis. `<br>` is a line break and has no closing tag.

Assignment 4: Divs, Attributes and a Small Fix

Aim: Group content with divs, use attributes, and fix errors.

Key ideas. A `div` is a generic block container. A `class` can be reused on many elements; an `id` must be unique on a page. A `data-*` attribute stores extra information in the HTML and is not shown on the page.

Task 1, layout.html. Three divs with the classes header, main and footer, each with some text. They appear one under the other.

Task 2, attributes.html.

```
<div id="profile">
  <p data-topic="hobbies">I enjoy reading, cricket and coding.</p>
</div>
```

Open the browser's Elements panel (F12) to see `data-topic` on the paragraph.

Task 3, attendance-fixed.html. The broken page had three errors:

1. `<h1>Attendance Report` was never closed. It needs `</h1>`.
2. `<p>Asha: 92%<p>` opened a second paragraph instead of closing the first.
3. The correct version is `<p>Asha: 92%</p>`.

Fixed body:

```
<h1>Attendance Report</h1>
<p>Asha: 92%</p>
<p>Rohan: 85%</p>
```

Assignment 5: Links and Images

Aim: Link two pages and add a picture with alt text.

Key ideas. The anchor tag `<a href="...">` creates a link. A relative path such as `portfolio.html` works when both files are in the same folder. The `alt` attribute describes an image for screen readers and shows if the image is missing.

Task 1, home.html. Heading Welcome to My Site and `<p>Visit my <a href="portfolio.html">portfolio</a>.</p>`.

Task 2, portfolio.html. A heading and `<a href="home.html">Go back to the home page</a>`.

Task 3, portfolio-image.html.

```
<img src="images/photo.jpg" width="200" height="200" alt="A photo from my
portfolio">
```

Replace `photo.jpg` with the file name your instructor gives you and keep the picture in an `images` folder next to the page.

Assignment 6: Types of HTML Lists

Aim: Use unordered, ordered, nested and description lists.

Key ideas. `<ul>` makes bullets, `<ol>` makes numbers, and each item is an `<li>`. A nested list goes inside an `<li>`, before its closing tag. A description list uses `<dl>`, with `<dt>` for the term and `<dd>` for the description.

Task 1, shopping-list.html. Three bullets: Notebook, Pen, Bag.

Task 2, canteen-menu.html.

```
<ol>
  <li>Samosa</li>
  <li>Sandwich
    <ol>
      <li>Cheese</li>
      <li>Vegetable</li>
    </ol>
  </li>
</ol>
```

The inner list is indented and restarts its numbering at 1.

Task 3, faq.html. One `<dl>` with two `<dt>` questions, each followed by a `<dd>` answer.

Assignment 7: About Me Page

Aim: Use semantic tags, in-page links and a mailto link.

Key ideas. Semantic tags describe their purpose: `header`, `nav`, `main`, `section`, `footer`. A link to `#about` jumps to the element with `id="about"`. A `mailto:` link opens the email program.

```
<header>
  <h1>Your Name</h1>
  <nav><a href="#about">About</a> <a href="#contact">Contact</a></nav>
</header>
<main>
  <section id="about"><h2>About me</h2><p>...</p></section>
</main>
<footer id="contact">
  <p>Email me: <a href="mailto:you@example.com">you@example.com</a></p>
</footer>
```

Clicking Contact scrolls to the footer because its id is `contact`. The three tasks build this one file step by step, so submit the file after each step or the final one, as your instructor asks.

Assignment 8: Sign-Up Forms and Choices

Aim: Build forms with labelled fields, choices and validation.

Key ideas. The `form` tag sends data using `method` and `action`. A `label` with `for="x"` is linked to the input with `id="x"`, so clicking the label focuses the field. `type="password"` hides characters and `type="email"` checks the email format.

Task 1, signup.html. `<form method="post" action="thanks.html">` with three label and input pairs: name, email, password.

Task 2, choices.html. Two radio buttons share `name="gender"`, which is why only one can be picked. The dropdown is a `<select>` with two `<option>` tags: BCA and B.Tech.

Task 3, signup-required.html. The inputs have `required`, and the name field has `placeholder="Enter your full name"`. A `<button type="submit">Submit</button>` is added. If you click Submit with an empty field, the browser shows its own message.

Assignment 9: Tables with Headers and Footers

Aim: Build structured tables with caption, header, body, footer and scope.

Key ideas. `<table>` contains `<caption>`, `<thead>`, `<tbody>` and `<tfoot>`. Each row is a `<tr>`; header cells are `<th>` and data cells are `<td>`. The `scope` attribute tells screen readers whether a header applies to a column or a row.

Task 1, marks.html. Caption Marks of Class A, header Name, Maths, Science, and rows for Asha 78 85, Rohan 64 70, Neha 91 88.

Task 2, marks-average.html. Only Asha and Rohan, with a footer row:

- Maths average = (78 + 64) / 2 = 71
- Science average = (85 + 70) / 2 = 77.5

```
<tfoot><tr><td>Class average</td><td>71</td><td>77.5</td></tr></tfoot>
```

Task 3, marks-accessible.html. Same table without the footer. Each header cell gets `scope="col"` and each name cell gets `scope="row"`. Nothing changes visually; check the Elements panel to see the attributes.

Assignment 10: CSS Fundamentals

Aim: Apply CSS in three ways and use selectors.

Key ideas. CSS can be inline (a `style` attribute, affects one element), internal (a `<style>` block in the head, affects one page) or linked (a separate `.css` file, shared by many pages). Selectors: `h1` selects a tag, `.note` selects a class, `#title` selects an id.

Task 1, inline-internal.html.

```
<style>
  h1 { color: navy; }
  .highlight { background-color: yellow; }
</style>
...
<p style="color: red;">Inline style</p>
<p class="highlight">Highlighted</p>
```

Task 2, style.css.

```
h1 { color: navy; }
.note { font-style: italic; }
#title { text-decoration: underline; }
p { font-size: 18px; color: grey; }
```

Task 3, linked.html. It contains `<link rel="stylesheet" href="style.css">` in the head, an `h1` with `id="title"` and two paragraphs with `class="note"`. It only looks styled when style.css is in the same folder.

Assignment 11: Units and CSS Variables

Aim: Use CSS units and custom properties.

Key ideas. `px` is a fixed size. `%` is relative to the parent element. `rem` is relative to the root font size (usually 16px), so 1.5rem is about 24px. A CSS variable is defined with `--name: value` and used with `var(--name)`. Defining variables in `:root` makes them available everywhere.

Task 1, units.html. One div with `width: 200px; height: 40px`, one with `width: 50%; height: 40px`, both with background colours, and a paragraph with `font-size: 1.5rem`. Resize the window: the 200px box stays the same, the 50% box changes.

Task 2, variables.html.

```
:root {
  --main-color: #1d3557;
  --accent-color: #e63946;
  --page-padding: 20px;
}
body { color: var(--main-color); padding: var(--page-padding); }
h1 { color: var(--accent-color); }
```

Task 3, tokens.css. Variables `--primary-color`, `--accent-color`, `--heading-size: 2rem` and `--spacing: 16px`, used for the body colour and padding and for the h1 colour and size. The extra file tokens-demo.html links it so you can see the result. The benefit of variables is that changing one value updates the whole site.

Assignment 12: Box Model, Cards and Scrolling

Aim: Understand box sizing, style a card, and make a box scroll.

Key ideas. Every element is a box made of content, padding, border and margin. With `content-box` the width applies to the content only, so the total width is larger. With `border-box` the width includes padding and border.

Task 1, box-model.html. Both boxes have `width: 200px; padding: 20px; border: 5px solid`.

- content-box total width = 200 + 20 + 20 + 5 + 5 = 250px
- border-box total width = 200px

So the first box looks wider. Check this in the browser's box model view.

Task 2, card.html.

```
.card { background-color: #e0f2fe; border: 2px solid #0369a1; border-radius:
12px; padding: 16px; }
```

It contains an `h3` and a paragraph. `border-radius` rounds the corners.

Task 3, scroll.html. A div with `height: 120px; overflow: auto;` holding five paragraphs, Line 1 to Line 5. The text is taller than the box, so a scrollbar appears.

Assignment 13: Flexbox Layouts

Aim: Lay out items in rows using Flexbox.

Key ideas. `display: flex` makes a container lay out its children in a row. `justify-content` aligns along the main axis (horizontal by default). `align-items` aligns along the cross axis (vertical). `flex-wrap: wrap` lets items move to a new row when there is no space. `gap` sets spacing between items.

Task 1, nav-flex.html. `nav { display: flex; gap: 20px; background-color: #222; }` with white links for Home, About, Events and Contact.

Task 2, centred-cards.html.

```
.row { display: flex; justify-content: center; align-items: center; gap:
20px; height: 250px; }
```

Three cards (Starter 199, Standard 349, Premium 499) sit in one row, centred horizontally and vertically. The row has a fixed height so that vertical centring is visible.

Task 3, wrapping-cards.html. Heading Stationery Shop and a row with `display: flex; flex-wrap: wrap; gap: 16px`. The four cards are Notebook 60, Pen 15, Bag 350 and Book 120, each `width: 150px`. Narrow the window and the cards drop onto new rows.

Assignment 14: CSS Grid Layouts

Aim: Build two-dimensional layouts with Grid.

Key ideas. `display: grid` creates rows and columns. The `fr` unit is a share of the free space. `repeat(auto-fill, minmax(150px, 1fr))` fits as many columns of at least 150px as there is room for. `grid-template-areas` lets you name regions and place elements with `grid-area`.

Task 1, grid-first.html. `grid-template-columns: 1fr 1fr 1fr; gap: 12px;` with six boxes numbered 1 to 6, giving two rows of three.

Task 2, gallery.html. Heading Club Photo Gallery and six boxes Photo 1 to Photo 6 with `grid-template-columns: repeat(auto-fill, minmax(150px, 1fr)); gap: 10px;`. A wider window fits more boxes per row.

Task 3, dashboard-areas.html.

```
.dashboard {
  display: grid;
  grid-template-columns: 200px 1fr;
  grid-template-areas:
    "header header"
    "sidebar main"
    "footer footer";
}
.header { grid-area: header; }
.sidebar { grid-area: sidebar; }
.main { grid-area: main; }
.footer { grid-area: footer; }
```

The header and footer span both columns. The sidebar is a fixed 200px and the main area takes the rest.

Assignment 15: Responsive Pages with Media Queries

Aim: Make pages adapt to screen size.

Key ideas. The viewport meta tag `<meta name="viewport" content="width=device-width, initial-scale=1">` makes phones use their real width. Mobile-first means writing the phone styles first, then adding `@media (min-width: 600px)` for larger screens. `max-width: 100%` stops an image from overflowing its container.

Task 1, mobile-first.html. Base styles stack the two cards. Then:

```
@media (min-width: 600px) {
  .cards { display: flex; gap: 12px; }
  .card { flex: 1; }
}
```

Below 600px the cards are stacked; at 600px and up they sit side by side.

Task 2, responsive-image.html. `img { max-width: 100%; height: auto; }`. On a narrow window the picture shrinks and keeps its proportions. On a wide window it stays at its own size. Use the file name your instructor gives you.

Task 3, responsive-text.html.

```
body { font-size: 1rem; }
h1 { font-size: 1.5rem; }
@media (min-width: 600px) { h1 { font-size: 2.5rem; } }
```

At 600px and wider the heading grows from 1.5rem to 2.5rem.

How to submit

1. Open each HTML file in a browser and take a screenshot for every task. Use the browser's developer tools (F12) and the device toolbar to test Assignments 13 to 15.
2. Copy the terminal output for Assignments 1 and 2.
3. Check that every file has your name, roll number and assignment number at the top.
4. Submit the files named in each assignment's Submit line.